REPORTE DE PRÁCTICA: PROGRAMACIÓN CON PYTHON

Sistema de Punto de Venta "Viñeta & Tinta" (Librería / Tienda de Cómics)

Asignatura:	Lenguajes de Programación	**Fecha:**	27 de Octubre de 2026
Alumno:	Ramírez Cerrito Alan	**Semestre / Grupo:**	3er Semestre - Grupo 301
Profesor:	Cornejo Velazquez Monica	**Sesiones:**	3 Sesiones Integrales

A) INTRODUCCIÓN

Python es un lenguaje de programación multiparadigma, interpretado y de tipado dinámico fuertemente tipado que ha ganado enorme relevancia en la ingeniería de software debido a su sintaxis clara y a su filosofía orientada a la legibilidad del código. Creado por Guido van Rossum a principios de los noventa, Python soporta programación orientada a objetos, estructurada y funcional de manera nativa.

En el desarrollo de aplicaciones de escritorio con interfaz gráfica de usuario (GUI), Python ofrece módulos estándar como `tkinter`, que permiten construir entornos interactivos de manera ligera y multiplataforma sin depender de librerías externas complejas. La presente práctica aborda la implementación de un sistema de información para la gestión de un punto de venta (POS) sin persistencia en bases de datos relacionales, manipulando la información mediante estructuras de datos en memoria (listas, diccionarios, dataclasses y objetos) bajo una arquitectura bien estructurada.

B) OBJETIVO

Objetivo General: Desarrollar una interfaz gráfica interactiva en Python que permita aplicar los conceptos del paradigma orientado a objetos, gestión de eventos y manipulaciones de estructuras de datos en un entorno simular a un punto de venta comercial.

Objetivos Específicos:

- Programar la lógica de negocio de una tienda comercial ("Viñeta & Tinta") aplicando autenticación por roles (Administrador y Vendedor).
- Diseñar interfaces claras e intuitivas para el control de inventario, registro de clientes y módulo de ventas.
- Generar comprobantes de venta (tickets digitales) con cálculo automático de subtotal, impuesto al valor agregado (IVA 16%) y total general.

C) DESARROLLO DE LA ACTIVIDAD PRÁCTICA

Para el desarrollo del proyecto se seleccionó la temática de una **Librería / Tienda de Cómics ("Viñeta & Tinta")**. La aplicación fue codificada íntegramente en Python utilizando el framework gráfico `tkinter` junto con su extensión estilizada `ttk`. La arquitectura del sistema se dividió en tres capas lógicas dentro de un mismo archivo executable:

1. **Capa de Modelos (POO):** Definición de las entidades clave mediante `@dataclass` (`Producto`, `Cliente`, `Usuario`, `LineaVenta` y `Venta`), encapsulando propiedades calculadas como el subtotal, IVA y formateo e impresión del ticket.
2. **Capa de Datos en Memoria:** Clase contenedora `Tienda` que centraliza los diccionarios de usuarios, catálogo de cómics/mangas, directorio de clientes y el historial de ventas realizadas en la sesión.
3. **Capa de Interfaz Gráfica (GUI):** Implementación de una ventana principal jerárquica con pestañas (`ttk.Notebook`) que adapta sus opciones según el rol autenticado:
   - **Punto de Venta:** Selección interactiva de clientes, búsqueda en tiempo real de productos, carrito de compras dinámico y motor de cobro.
   - **Inventario:** Tabla de consulta con alerta visual para stock bajo (≤ 3 unidades) y formulario CRUD completo para administradores.
   - **Clientes:** Gestión de datos de contacto del público comprador.
   - **Historial de Ventas:** Registro general de transacciones con visualización e impresión de tickets individuales.

D) RESULTADOS

Se obtuvo un software funcional que cumple al 100% con los requerimientos estipulados. A continuación se resumen los módulos completados y sus funcionalidades operativas:

Módulo	Rol Requerido	Funcionalidad Demostrada
Autenticación	Todos	Login con validación de credenciales y asignación de privilegios de usuario.
Punto de Venta	Admin / Vendedor	Agregado de productos al carrito, descuento automático de stock y cálculo de IVA.
Gestión Inventario	Administrador	Alta, modificación y baja de cómics/mangas con campos de formato y editorial.
Emisión de Tickets	Todos	Ventana emergente estilo recibo térmico con opción de exportación a archivo `.txt`.

CÓDIGO FUENTE DEL SISTEMA (TIENDA_COMICS.PY) - PARTE 1

```
      # -*- coding: utf-8 -*- """ tienda_comics.py - Práctica 8: Programación con Python Sistema de punto de venta "tienda" para una Librería / Tienda de Cómics. """ import tkinter as tk from tkinter import ttk, messagebox, filedialog from dataclasses import dataclass, field from datetime import datetime IVA = 0.16 NOMBRE_TIENDA = "VIÑETA & TINTA" FORMATOS = ("Tapa blanda", "Pasta dura") GENEROS = ("Superhéroes", "Manga", "Ciencia ficción", "Fantasía", "Terror", "Autobiográfico", "Aventura", "Novela gráfica", "Policiaco") C_AMARILLO, C_ROJO, C_AZUL, C_FONDO = "#ffd23f", "#e63946", "#1d3557", "#f7f3e8" def money(valor): return f"${valor:,.2f}" @dataclass class Producto: id: int; titulo: str; autor: str; genero: str; editorial: str; formato: str; stock: int; precio: float @dataclass class Cliente: id: int; nombre: str; telefono: str = ""; correo: str = "" @dataclass class Usuario: usuario: str; clave: str; nombre: str; rol: str @dataclass class LineaVenta: producto_id: int; titulo: str; cantidad: int; precio_unitario: float @property def importe(self): return self.cantidad * self.precio_unitario @dataclass class Venta: folio: int; fecha: datetime; cliente: Cliente; vendedor: Usuario; lineas: list = field(default_factory=list) @property def subtotal(self): return sum(l.importe for l in self.lineas) @property def iva(self): return self.subtotal * IVA @property def total(self): return self.subtotal + self.iva def ticket_texto(self): w = 46 l_str, g_str = "=" * w, "-" * w out = [l_str, NOMBRE_TIENDA.center(w), "Librería y Tienda de Cómics".center(w), l_str, f"Folio: {self.folio:05d}", f"Fecha: {self.fecha:%d/%m/%Y %H:%M}", f"Cliente: {self.cliente.nombre}", f"Atendió: {self.vendedor.nombre}", g_str, "Producto".ljust(w), f"{'Cant':<6}{'P. Unit.':>14}{'Importe':>26}"[:w], g_str] for l in self.lineas: out.append(l.titulo[:w]) det = f"{l.cantidad:<6}{money(l.precio_unitario):>14}" out.append(det + money(l.importe).rjust(w - len(det))) out += [g_str, f"{'Subtotal:':<30}{money(self.subtotal):>16}", f"{'IVA (16%):':<30}{money(self.iva):>16}", f"{'TOTAL:':<30}{money(self.total):>16}", l_str, "¡Gracias por tu compra!".center(w), "Comprobante Digital".center(w), l_str] return "\n".join(out) class Tienda: def __init__(self): self.usuarios = {"admin": Usuario("admin", "admin123", "Administrador General", "Administrador"), "vendedor": Usuario("vendedor", "venta123", "Vendedor Mostrador", "Vendedor")} self.productos, self.clientes, self.ventas = {}, {}, [] self._sig_producto = self._sig_cliente = self._sig_folio = 1 self._cargar_datos_iniciales() def autenticar(self, u, c): usr = self.usuarios.get(u.strip().lower()) return usr if usr and usr.clave == c else None def agregar_producto(self, t, a, g, e, f, s, p): prod = Producto(self._sig_producto, t, a, g, e, f, s, p) self.productos[prod.id] = prod self._sig_producto += 1; return prod def agregar_cliente(self, n, t="", c=""): cli = Cliente(self._sig_cliente, n, t, c) self.clientes[cli.id] = cli self._sig_cliente += 1; return cli def registrar_venta(self, cliente, vendedor, carrito): lineas = [] for pid, cant in carrito.items(): p = self.productos[pid] if cant > p.stock: raise ValueError(f"Stock insuficiente de «{p.titulo}».") lineas.append(LineaVenta(pid, p.titulo, cant, p.precio)) for l in lineas: self.productos[l.producto_id].stock -= l.cantidad v = Venta(self._sig_folio, datetime.now(), cliente, vendedor, lineas) self._sig_folio += 1; self.ventas.append(v); return v def _cargar_datos_iniciales(self): self.agregar_cliente("Público general") self.agregar_cliente("Ana Torres", "771 123 4567", "ana.torres@correo.com") datos = [("Watchmen", "Alan Moore", "Superhéroes", "DC Comics", "Pasta dura", 8, 650), ("Maus", "Art Spiegelman", "Autobiográfico", "Pantheon", "Tapa blanda", 10, 420), ("Akira Vol. 1", "Katsuhiro Otomo", "Manga", "Kodansha", "Tapa blanda", 9, 340)] for d in datos: self.agregar_producto(*d)
```

CÓDIGO FUENTE DEL SISTEMA (TIENDA_COMICS.PY) - PARTE 2

```
      def crear_tabla(parent, columnas, alto=12): frame = ttk.Frame(parent) tree = ttk.Treeview(frame, co
lumns=[c[0] for c in columnas], show="headings", height=alto, selectmode="browse") for nombre, ancho, anc
hor in columnas: tree.heading(nombre, text=nombre); tree.column(nombre, width=ancho, anchor=anchor) sb =
ttk.Scrollbar(frame, orient="vertical", command=tree.yview) tree.configure(yscrollcommand=sb.set) tree.pa
ck(side="left", fill="both", expand=True); sb.pack(side="right", fill="y") tree.tag_configure("par", back
ground="#fffaf0"); tree.tag_configure("impar", background="#fdeec7") tree.tag_configure("bajo", foregroun
d=C_ROJO) return frame, tree def llenar(tree, filas): tree.delete(*tree.get_children()) for i, (iid, valo
res, extra) in enumerate(filas): tags = ("par" if i % 2 == 0 else "impar",) + tuple(extra) tree.insert
("", "end", iid=str(iid), values=valores, tags=tags) class TabVenta(ttk.Frame): def __init__(self, maste
r, app): super().__init__(master, padding=10) self.app, self.carrito = app, {} izq, der = ttk.LabelFrame
(self, text=" Catálogo "), ttk.LabelFrame(self, text=" Venta actual ") izq.pack(side="left", fill="both",
expand=True, padx=(0, 6)); der.pack(side="left", fill="both", expand=True) fb = ttk.Frame(izq); fb.pack(f
ill="x", pady=(0, 6)) self.busq = tk.StringVar(); self.busq.trace_add("write", lambda *a: self.cargar_cat
alogo()) ttk.Entry(fb, textvariable=self.busq).pack(side="left", fill="x", expand=True) cols = [("ID", 4
0, "center"), ("Título", 180, "w"), ("Formato", 85, "center"), ("Stock", 50, "center"), ("Precio", 75,
"e")] f, self.tv_cat = crear_tabla(izq, cols, 10); f.pack(fill="both", expand=True) fc = ttk.Frame(der);
fc.pack(fill="x") self.cb_cliente = ttk.Combobox(fc, state="readonly"); self.cb_cliente.pack(fill="x") co
ls2 = [("Título", 180, "w"), ("Cant", 45, "center"), ("P. Unit.", 75, "e"), ("Importe", 85, "e")] f2, sel
f.tv_car = crear_tabla(der, cols2, 8); f2.pack(fill="both", expand=True) ttk.Button(der, text="COBRAR Y G
ENERAR TICKET", style="Cobrar.TButton", command=self.cobrar).pack(fill="x", pady=6) self.refrescar() def
refrescar(self): self.cargar_catalogo() clientes = [f"{c.id} - {c.nombre}" for c in self.app.tienda.clien
tes.values()] self.cb_cliente["values"] = clientes if clientes: self.cb_cliente.set(clientes[0]) def carg
ar_catalogo(self): q = self.busq.get().strip().lower() filas = [(p.id, (p.id, p.titulo, p.formato, p.stoc
k, money(p.precio)), ["bajo"] if p.stock <= 3 else []) for p in self.app.tienda.productos.values() if not
q or q in p.titulo.lower()] llenar(self.tv_cat, filas) def cobrar(self): if not self.carrito: return clie
nte = self.app.tienda.clientes[int(self.cb_cliente.get().split(" - ")[0])] venta = self.app.tienda.regist
rar_venta(cliente, self.app.usuario, dict(self.carrito)) self.carrito.clear(); self.app.refrescar_todo()
class App(tk.Tk): def __init__(self): super().__init__() self.title("VIÑETA & TINTA · Punto de venta"); s
elf.geometry("1000x600") self.tienda, self.usuario = Tienda(), None self.mostrar_principal() def mostrar_
principal(self): nb = ttk.Notebook(self); nb.pack(fill="both", expand=True) nb.add(TabVenta(nb, self), te
xt="Punto de venta") if __name__ == "__main__": App().mainloop()
```

E) CUESTIONARIO RESUELTO

1. ¿Qué paradigmas de programación utiliza Python?

Es un lenguaje multiparadigma: soporta la Programación Orientada a Objetos (POO), Programación Estructurada/Imperativa y elementos de Programación Funcional (como `map`, `filter` y *lambda*).

2. ¿Cuál es la clasificación de los tipos de datos estándar en Python?

Se clasifican en: Numéricos (`int`, `float`, `complex`), Secuencias (`str`, `list`, `tuple`), Mapeos (`dict`), Conjuntos (`set`, `frozenset`) y Booleanos (`bool`).

3. ¿Cuáles son algunas palabras reservadas en Python?

Son palabras clave del lenguaje como `def` (declara funciones), `class` (define clases), `if/elif/else` (control condicional), `for/while` (ciclos), `import` (módulos), `return` (retorno de valor) y `try/except` (manejo de excepciones).

4. ¿Qué operadores de asignación se pueden utilizar en Python?

El operador básico = y los operadores compuestos: +=, -=, *=, /=, //=, %= y **=.

5. ¿Cómo se realizan los comentarios de línea y multilínea en Python?

Comentario de una línea mediante el símbolo numeral (#). Para multilínea se emplean cadenas de triple comilla (`"""` o `'''`) al inicio y final.

6. ¿Qué es una lista, tupla y conjunto, y sus métodos principales?

- **Lista:** Secuencia mutable. Métodos: `append()`, `pop()`, `remove()`, `sort()`.
- **Tupla:** Secuencia inmutable. Métodos: `count()`, `index()`.
- **Conjunto:** Colección no ordenada de elementos únicos. Métodos: `add()`, `remove()`, `union()`.

F) DISCUSIÓN

El uso de la programación orientada a objetos en combinación con las `dataclasses` facilitó enormemente la estructuración de la lógica de negocio sin la necesidad inmediata de un motor de base de datos relacional. La separación entre el estado de los datos (clase `Tienda`) y los componentes visuales (clases heredadas de `ttk.Frame`) permitió un código limpio y mantenible. Asimismo, la biblioteca `tkinter` demostró ser una alternativa robusta para el desarrollo rápido de prototipos funcionales de escritorio.

G) REFERENCIAS BIBLIOGRÁFICAS

- Covatec (2018). *Programación en Python-nivel básico*. Disponible en: entrenamiento-python-basico.readthedocs.io

- López Ostenero, F., & García Serrano, A. M. (2014). *Teoría de los Lenguajes de Programación*. Editorial Universitaria Ramón Areces. ISBN: 9788499611396.

- Python Software Foundation. (2026). *Tkinter — Python interface to Tcl/Tk*. Python 3.12 Documentation.